

Energy-Efficient Time Series Applications on IoT Devices via Sensor-Aware Early Exit Random Forest Architectures

Sirine Belhaj*, *Student Member, IEEE*, Negar Haghpahanhi*, *Student Member, IEEE*, Lubah Nelson*, *Student Member, IEEE*, Dina Hussein, *Member, IEEE*, and Ganapati Bhat, *Member, IEEE*

Abstract—Time-series data processing is critical in several high-impact applications including mobile health and environmental monitoring. Energy efficiency is a critical bottleneck for continuous time-series classification on edge internet of things (IoT) devices, especially in wearable health monitoring, where battery constraints are severe. Most applications and machine learning (ML) models for time-series processing on IoT devices wait for complete sensor data windows, which increases energy consumption of the device. To address this, we introduce Sensor-aware Early-Exit Random Forests (SEE-RF) – the first purpose-built random forest (RF) architecture with natively integrated early-exit mechanisms into RFs for joint sensing and inference optimization. By dynamically stopping the inference process once sufficient classification confidence is reached, SEE-RF significantly reduces both sensing and computational energy. More specifically, the proposed models use low power mode for sensors as soon as sufficient confidence is reached so that the system achieves significant sensing energy savings. Across six diverse health datasets, SEE-RF achieves significant sensing energy savings with negligible loss in accuracy. End-to-end measurements on a Raspberry Pi Zero 2W embedded device show that the proposed SEE-RFs achieve 29–42% energy savings in real-world settings with two sensors.

Index Terms—Early exits, energy-efficient inference, time-series classification, wearable health, embedded systems.

I. INTRODUCTION

The deployment of edge artificial intelligence (AI) across billions of Internet of Things (IoT) devices is creating a growing energy challenge that threatens both the sustainability and accessibility of AI-enabled services [1, 2]. The edge-AI energy efficiency challenges manifest across multiple applications, from smart agriculture sensors monitoring crop health to industrial IoT predicting equipment failures [3–5]. However, it becomes most acute and consequential in wearable health monitors that run continuous time-series inference with small batteries [6–9]. For example, for the 422 million people with diabetes requiring continuous glucose monitoring, or the 1.3 billion with hypertension needing regular tracking, battery life determines whether AI-powered health monitoring is practical

or aspirational [10, 11]. Consequently, optimizing the energy consumption of wearable devices is crucial.

Modern wearable devices for health applications typically include multi-modal sensor fusion and processing [3, 12]. The data are processed by machine learning (ML) and AI algorithms to make inference for the health application. Current systems typically keep every sensor active until complete data windows arrive. This leads to sensing power dominating the energy budget [12, 13], which further leads to frequent battery recharges and low compliance by users [3, 14–16].

The energy challenges arise from a fundamental inefficiency in current time-series classification systems: they consume complete sensor windows and execute full inference pipelines regardless of classification difficulty. This “data-oblivious” approach directly conflicts with the resource constraints of battery-powered edge devices and represents a source of preventable energy waste. This waste has two separable components: *sensing energy* from keeping sensors active during data collection, and *computational energy* from executing the ML model. Prior work has addressed each in isolation, leaving joint architectural optimization as an open problem. Therefore, there is a strong need to investigate approaches that adaptively tune the sensor operation based on difficulty of the input data.

Prior methods have addressed sensing and computational energy in isolation. Compute-only early-exit methods [17–19] reduce inference cost but require complete sensor windows. Early time-series classification methods [20, 21] classify sooner but do not control sensor power states. While joint sensing-compute optimization has been explored using early exits with partial sensor windows, existing Random Forest (RF) implementations rely on independently-trained classifiers per data fraction without modifying the RF architecture itself. The problem of natively integrating early exits into RF structure thus enabling training-time feature constraints and cross-exit learning within a single model remains an open challenge.

This paper introduces Sensor-Aware Early-Exit Random Forests (SEE-RF), which address this open problem through two purpose-built RF architectures. Natively integrating early exits into RF structure requires solving three problems: (1) standard RFs have no mechanism to constrain which features a tree node may access at a given depth; (2) independently-trained classifiers per data fraction share no learned information across exits; and (3) applying a stopping criterion to a pre-trained RF saves compute but not sensing energy, since data collection has already completed. SEE-RF addresses all three through architectural redesign of the RFs.

*S. Belhaj, N. Haghpahanhi, and L. Nelson contributed equally to this work.

S. Belhaj, N. Haghpahanhi and G. Bhat are with the School of Electrical Engineering and Computer Science, Washington State University, Pullman, WA, 99164 USA e-mail: (sirine.belhaj@wsu.edu, negar.haghpahanhi@wsu.edu, ganapati.bhat@wsu.edu). D. Hussein was with the School of Electrical Engineering and Computer Science, Washington State University. She is currently with the School of Medicine, Wake Forest University. L. Nelson was with the School of Electrical Engineering and Computer Science, Washington State University. This work was supported, in part, by the National Science Foundation (NSF) CAREER Award CNS-2238257 and the WSU Office of Research through the President’s Big Idea’s Initiative (26-PBII-T2-01).

We choose RFs as the base architecture for three reasons that make them uniquely suited to this problem. First, RFs execute efficiently on microcontroller (MCU)-class hardware without specialized accelerators, unlike neural networks that require multiply-accumulate hardware or GPU support [17, 18]. Second, RF ensemble vote distributions provide well-calibrated confidence estimates naturally [19], yielding low Expected Calibration Error (ECE) without post-hoc recalibration steps. This is essential for exit decisions: a poorly-calibrated model exits too early on uncertain predictions, degrading accuracy. Third, the hierarchical structure of decision trees, where depth encodes increasing feature complexity maps naturally onto progressive data introduction. Each exit level sees incrementally more of the sensor window. While these properties make RFs the ideal first target, the proposed early-exit architectures are structurally agnostic to the base ensemble method and can be extended to other tree-based variants.

We investigate two complementary RF architectures: Vertical and Sequential SEE-RF. Vertical SEE-RF (SEE-vRF) architecture splits the trees in each RF vertically where each level corresponds to a portion of sensor data and early exit. Horizontal SEE-RF (SEE-hRF) employs cascaded sub-forests with a subset of trees in the RF assigned to each early exit.

We validate the proposed early-exit RF classifiers using six publicly available health datasets spanning activity recognition and electroencephalogram (EEG) processing. Evaluations show that the proposed RF models achieve significant sensing energy savings while maintaining accuracy levels comparable to baseline RFs that use full data windows. Moreover, comparisons to prior approaches [17, 20, 21] demonstrate that the proposed approach outperforms them in energy efficiency, highlighting the advantages of SEE-RFs.

We also perform real-world validations to assess feasibility of SEE-RFs. First, we implement the SEE models on the Raspberry Pi Zero 2W [22] device to understand computational needs of the RF models and time taken to process each exit. The measurements show that the proposed SEE-RFs add less than 5% overhead when compared to default RFs. We also create a complete system with Raspberry Pi Zero 2W and two sensors to emulate real-world deployment. Following the execution traces from one of the datasets, the measurements show that the SEE-RFs lead up to 29–42% energy savings, which aligns with the estimates from our simulations.

In summary, this paper makes the following contributions:

- SEE-vRF: A vertically-split RF architecture that constrains tree nodes at each depth level to only the features available at the corresponding temporal exit, enabling a single tree to produce valid predictions at multiple points during data collection.
- SEE-hRF: A cascaded sub-forest architecture with cross-forest weight propagation, where sample weights propagate across sub-forest boundaries, so that later exits learn from earlier exits' mistakes rather than training in isolation. This cross-exit information flow is new for RFs.
- Comprehensive evaluation with six datasets and embedded system boards along with end-to-end implementation.

Source Code Release: Datasets, code, and models for SEE-RFs are available at: <https://github.com/gmbhat/see-rf>.

II. RELATED WORK

Prior work on energy-efficient inference for IoT edge devices has addressed sensing and computational energy largely in isolation [17, 23, 24]. Early-exit architectures for neural networks have demonstrated computational savings in resource-constrained environments. BranchyNet [23] utilizes intermediate classifiers at multiple network depths, enabling inference termination when prediction confidence exceeds learned thresholds. MSDNet [24] extends this paradigm through multi-scale feature processing for anytime prediction under computational budgets. These architectures achieve up to $5\times$ reduction in computational cost while maintaining competitive accuracy. Adaptive mobile inference methods such as AdaDeep [25] and NestDNN [26] further reduce compute through model compression and pruning under runtime resource constraints. All of these methods share a fundamental limitation: complete sensor data acquisition must occur before inference begins. In wearable edge devices where sensing power can constitute 60–80% of total system energy [12, 13], compute-only savings address the smaller component of the energy budget.

A complementary line of work reduces sensing energy without addressing inference cost [20, 21]. Adaptive sampling approaches [27, 28] throttle sensor sampling rates to conserve energy, but degrade accuracy uniformly across all classes by discarding data indiscriminately. Trapeznikov and Saligrama [29] address sequential sensor selection under budget constraints, querying increasingly complex sensors until prediction confidence is achieved. While conceptually related, their formulation selects discrete sensors in sequence rather than deciding *when* to stop reading a continuous temporal window. Moreover, the approach in [29] is for offline settings where additional measurements can be taken with more complex sensors when low-overhead sensors do not yield sufficient information, such as in medical imaging. Learned sensor scheduling approaches [30] determine which sensors to activate across windows, complementary to SEE-RF's within-window temporal exit. We can leverage this in SEE-RF like architectures to enable further energy savings by using a subset of sensors with early exits. However, we leave this for future work and focus on early exits without sensor selection. Cost-aware ensemble methods such as CEGB [31] assign acquisition costs to individual features; however, their cost model operates at the feature level rather than the temporal sensing level, making direct comparison with our setting inappropriate.

Bridging these two lines of work, Hussein et al. [32] explore early exits with partial sensor windows to reduce both sensing and computational energy. That work contributes architecturally integrated late-input blocks for CNNs. For RFs, however, no architectural modification is proposed; independently-trained standard classifiers at each data fraction are combined without modifying the RF structure. Moreover, independently trained RFs can lead to higher memory requirements since they do not share information between exits. In contrast, we propose modifications to the RF architecture to enable early exits with partial data. To the best of our knowledge, no existing work modifies the RF architecture itself to support joint sensing and computational energy optimization.

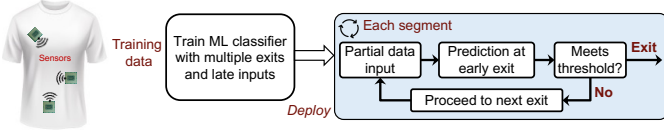


Fig. 1: An overview of both models, SEE-vRF and SEE-hRF, is presented. In both versions, once the classifier is trained, partial data is provided to each sub-forest. The exit condition is then evaluated; if it is satisfied, the process terminates early. Otherwise, the data proceeds to the next exit. This iterative process continues until the input exits.

III. SYSTEM OVERVIEW

We consider a system shown in Figure 1. We assume that multiple sensors on the body monitor physiological signals, such as body motion, heart rate, and blood oxygen [3]. Without loss of generality, we assume that a central processing unit aggregates data from all sensors for time-series processing. The data are used by the ML model to perform inference.

Training: The device aggregates data from all sensors along with the ground truth labels to generate training data. The key idea in the proposed framework is that not all time-series classification windows require the full sensor data and classification pipeline to achieve high accuracy. To this end, the proposed approach inserts multiple exit points in the classification pipeline with increasing amounts of data. The training block in Figure 1 applies to both proposed SEE-RF versions. Details on the training process for the two SEE-RF versions are provided in Sections V-A and V-B, respectively.

Deployment and Inference: Once trained, the ML models are deployed on the edge device for continuous health monitoring applications. Both RF versions collect data required for the first exit. Corresponding trees for either SEE-vRF or SEE-hRF are used to evaluate the class probabilities and confidence. If the confidence meets a pre-defined threshold, then the classifier exits and provides the class label. Sensors are transitioned into a low power mode until the beginning of next data window. Conversely, if the confidence does not meet the threshold, more data are collected to evaluate the next exit. For example, consider that the architecture has two early exits with 25% and 50% of data collected for each exit. In this case, the first sub-forest (exit) uses 0–25% of data, while the second sub-forest uses 0–50% of data. While it is possible to just use the new data, we see improved performance with cumulative data. This process continues until the classifier exits early or the terminal exit is reached.

IV. BACKGROUND AND PROBLEM SETUP

A. Time-Series Classification on Edge Devices

Edge time-series classification systems partition sensor streams into fixed-duration windows and execute inference on complete windows [4, 12, 33]. The energy profile of edge inference comprises two primary components: *sensing energy* E_{sense} consumed during data acquisition and *computational energy* $E_{compute}$ required for model execution. Sensors typically remain active throughout the entire window duration T , consuming energy proportional to sensor power $P_{sensor} \times T$ regardless of the intrinsic difficulty of the classification task. This *data-oblivious* approach creates substantial energy waste.

B. Random Forests for Edge Deployment

Time-series applications can utilize a wide variety ML models for performing inference at the edge. Among these we choose RFs for this paper since they present several architectural advantages that make them particularly suitable for resource-constrained edge environments. RFs offer the following key benefits for edge deployment:

Computational Simplicity: Tree traversal requires only comparison operations at each node. The worst-case inference complexity is $O(N_T \times D)$ where N_T denotes the number of trees and D represents maximum tree depth.

Hardware Agnosticism: RFs execute efficiently on general-purpose processors without specialized accelerators and provide inherent interpretability.

C. Architectural Design Choices for Early Exit RFs

Early-exit architectures must balance several competing objectives: prediction confidence, memory footprint, and deployment flexibility. We investigate two complementary approaches to address different RF deployment scenarios. Exit placement is temporal, not feature-based: each exit corresponds to an elapsed fraction of the sensor window duration, so exit k sees the first t_k seconds of all sensor channels. This is distinct from partitioning features into equal-sized groups.

- **Sensor-Aware Vertically Split RFs (SEE-vRF):** SEE-vRF splits trees in the RF vertically so that a subset of levels in each tree corresponds to each early exit. The root of the trees start with the first data portion and subsequent data are introduced after the early exit splits.
- **Sensor-Aware Horizontally Split RFs (SEE-hRF):** SEE-hRF splits RFs horizontally by employing a subset of trees for each early exit and portion of data. The predictions from each sub-forest refine predictions for future exits whenever an early exit is not taken for a given data window. The sequential nature of SEE-hRF helps the RF in combining predictions of all available sub-forests.

D. Problem Formulation for SEE-RFs

Consider a time-series classification task with window duration T , sampling frequency f_s , and C target classes. We define K potential exit points at times $\mathcal{T} = \{t_1, t_2, \dots, t_K\}$ where $0 < t_1 < t_2 < \dots < t_K = T$. Exits 1 to $K-1$ represent early exits while K is the default terminal exit. At each exit point $t_i \in \mathcal{T}$, the system observes sensor data $\mathbf{x}_{1:t_i} = p_i$ and must decide between two actions:

Exit inference with prediction $\hat{y}_i = f_{\theta}(\mathbf{x}_{1:t_i})$ and transition sensors to low-power mode.

Continue to next exit point t_{i+1} , collecting additional data.

Given this, the total energy consumption for a data window w that exits at time $t_i (1 \leq i \leq K)$ equals:

$$E_w(t_i) = \underbrace{P_{sensor} \times t_i}_{E_{sensor}(t_i)} + \underbrace{E_m(t_i)}_{E_{compute}(t_i)} + \underbrace{E_{sleep} + E_{wake}}_{E_{transition}}$$

where $E_m(t_i)$ represents the computational energy for evaluating exit i , and $E_{transition}$ captures sensor state-change overhead. The transition energy $E_{transition}$ includes sensor

sleep energy E_{sleep} and wake-up energy E_{wake} . The sensor wake-up time t_{wake} and stabilization time t_{stab} , both obtained from sensor datasheets. For the motion sensors used in our deployment, $t_{wake} + t_{stab} \approx 5$ ms, contributing less than 2% overhead relative to typical window durations of 1–5 seconds. Our objective is to minimize the expected energy consumption while maintaining classification performance:

$$\min_{\theta} \mathbb{E}_{\mathbf{X}, y \sim \mathcal{D}}[E(\mathbf{X})] \quad \text{s. t.} \quad \text{Acc}(\theta) \geq \text{Acc}_{baseline} - \epsilon$$

where θ denotes model parameters, $\mathcal{D}$ is the data distribution, and ϵ specifies acceptable accuracy degradation from baseline accuracy $\text{Acc}_{baseline}$.

Key Challenges for Joint Computational and Sensing Energy Optimization: One of the key goals for the SEE-RF architectures is to obtain sensing and computational energy savings with partial data windows. Vanilla RFs start the inference stage after collecting all necessary sensing data, thus increasing sensing energy. The proposed problem formulation deviates from this approach by progressively introducing more data at each exit. This creates a trade-off between sensing & computational energy and accuracy. Introducing more data at an early exit can improve accuracy while sacrificing energy savings. The problem can be thought of as a multi-objective problem with key variable being the data proportions at each exit, the size and depth of trees, and exit thresholds. The problem is challenging to solve natively through traditional optimization methods due to discrete nature of variables and the need to obtain accuracy/energy with each design.

E. Confidence Quantification via Entropy

The decision to exit or continue relies on quantifying confidence of the prediction at exit i . Following prior literature [23], we employ normalized entropy as our confidence metric:

$$\phi(\mathbf{p}) = -\frac{1}{\log C} \sum_{c=1}^C p_c \log p_c \quad (1)$$

where $\mathbf{p} = [p_1, \dots, p_C]$ represents the predicted class probability distribution. The normalization factor $\frac{1}{\log C}$ ensures $\phi(\mathbf{p}) \in [0, 1]$ regardless of the number of classes. The system accepts a prediction when $\phi(\mathbf{p}) < \tau_i$, where τ_i denotes the confidence threshold for exit i . Lower thresholds require higher confidence, leading to higher energy. Shannon entropy provides several advantages for exit decisions:

Uncertainty Quantification: Entropy naturally captures prediction uncertainty: low entropy indicates confident predictions while high entropy signals ambiguity.

Computational Efficiency: Computing entropy requires $O(C)$ operations, negligible compared to inference.

Calibration Quality via Expected Calibration Error. Because early-exit decisions are triggered by predicted confidence, the reliability of probability estimates directly affects both accuracy and energy consumption. To assess this reliability, we measure calibration using Expected Calibration Error (ECE) [34]. ECE partitions predictions into M equally

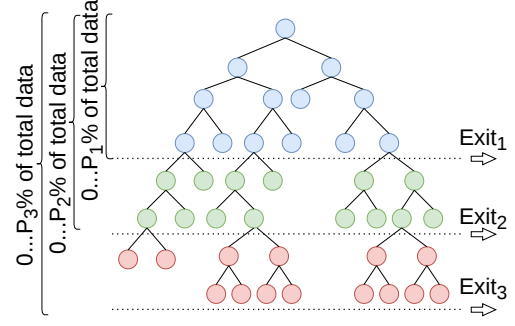


Fig. 2: Tree architecture in SEE-vRF

spaced confidence bins and computes the weighted average gap between empirical accuracy and predicted confidence:

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N_d} |\text{acc}(B_m) - \text{conf}(B_m)| \quad (2)$$

where B_m denotes samples whose confidence falls in bin m , N_d is number of data samples in a dataset, $\text{acc}(B_m)$ is the fraction correctly classified in that bin, and $\text{conf}(B_m)$ is the mean predicted confidence. Lower ECE indicates better alignment between confidence and true correctness. In our setting, poor calibration can cause premature exits (accuracy loss) or delayed exits (energy waste), making calibration essential for reliable entropy-based thresholds. We use $M = 10$ bins following standard practice [35].

V. PROPOSED SEE-RF MODELS

A. Vertically-Split Early Exit RFs (SEE-vRF)

The vertically split SEE-vRF splits each decision tree in the RF into multiple levels, where each level corresponds to an early exit with different percentages of data, as shown in Figure 2. The representative tree in the figure has three levels, with two early exits. Each exit level is shown with a different color for the nodes. The combination of early exits with partial sensor data ensures that SEE-vRF achieves savings in both sensing and computational energy. Key feature of SEE-vRF is that it splits each tree in the RF vertically so that only a subset of levels are executed when early exits are used.

SEE-vRF Training: Classical RFs are trained with the same weight given to all the class labels across all the trees. This method works when the entire tree is used for all classifications. However, the proposed RFs aim to classify easier labels in earlier exits, while harder classes are left for later exits. As such, we must fine-tune the weight assigned to each class as the trees are trained. To this end, we utilize Sequential RFs (sRF) that adapt samples and weights for classes to focus on more difficult classes as training progresses [36].

Following the typical sRF training procedure, we grow each tree sequentially so that each subsequent tree is influenced by the performance of previously trained trees. The training process starts by assigning uniform weight of $1/N$ to each data sample, where N is the number of training data samples. Once trained, the tree is tested on Out-Of-Bag (OOB) samples. The OOB data are samples that are not used to train a given tree, hence they provide unseen test data to analyze accuracy trees during training. Both SEE-RF versions use the same OOB

sample methodology to obtain accuracy. Using accuracy on the OOB samples, the weights are updated as follows:

$$w^{tr+1}(i) \propto w^{tr}(i) \cdot (1 + \mathbf{E}^{tr}(c)) \quad (3)$$

where $w^{tr}(i)$ is the weight of the i -th sample and $\mathbf{E}^{tr}(c)$ is the OOB misclassification rate for class c . Weights of correctly classified samples remain the same, while misclassified samples receive higher weights, ensuring future trees focus on harder examples in later exits.

Training of SEE-vRF uses the sample re-weighting described above with the following additional constraints due to unique architecture of SEE-vRF:

- Each tree must be split into multiple levels corresponding to the number of exits. Specifically, if the maximum tree depth is D , we assign $d_1, \dots, d_K$ levels for each exit where K is number of exits.
- The available data space is constrained at each level, i.e., each level can use features up to t_i when making node splitting decisions at the tree level d_i where t_i is the time duration for the data percentage available at level d_i .

Given the constraints, we start splitting the nodes from the root of each tree by using the data available before the first exit at time t_1 , i.e., $0 \dots P_1\%$ data. We include exit nodes at the end of the first split of tree which will serve to perform classification and check confidence later on during inference. We keep expanding the trees in a progressive tier-wise manner while incrementally adding new data, i.e. $0 \dots P_k\%$ for exit k and introducing new early exit points at the end of each tree portion. In other words, exit k utilizes cumulative data from $0 \dots P_k\%$ where $P_k\%$ denotes the percentage of the window data available for exit k . This process continues until either the maximum tree depth is reached, the number of samples at a node falls below the predefined minimum threshold, or the node achieves class purity. We account for early exits by adding a flag to the nodes at the end of each level to indicate that they correspond to exit nodes. These additional nodes use the information available up to the end of the level to provide classifications at that exit. Once the first tree is built, we calculate OOB misclassification rate for each class while treating it as a standard decision tree with no exits. The sample weights are then updated by increasing weights for wrongly classified samples. The next tree in the RF is trained in a similar manner with updated weights. Algorithm 1 summarizes the process of training the SEE-vRF models described above.

B. Horizontally-Split Early Exit RFs (SEE-hRF)

Our second architecture creates sub-forests in the RF to enable early exits. The horizontally-split early exit RF (SEE-hRF) framework is designed as a single model that includes multiple sequential trees in a cascaded architecture. This architecture utilizes a sequential adaptation of training samples and a weighted majority voting scheme across trees during training. However, typical RFs use a single exit with all data, which is not suitable for the proposed early exit approach. In contrast, our approach leverages sRF and structures the classification process into a series of interconnected sub-forests for early exits, as explained next.

Algorithm 1 Training SEE-vRF

Require: Dataset D , data percentages $Data\%$, tree splits $Splits$, number of estimators $N_{estimators}$, maximum tree depth max_depth , total number of windows N

Ensure: List of trained trees with vertical early-exits

- 1: $window_size \leftarrow$ length of input vector $D[0]$
- 2: Initialize sample weights $w(i) \leftarrow 1/N$ for all $i = 1, \dots, N$.
- 3: $Trees_list \leftarrow []$
- 4: **for** $t = 1$ to $N_{estimators}$ **do**
- 5: $D_{bag}, D_{oob} \leftarrow$ Weighted bootstrap sample and OOB from D
- 6: $current_nodes \leftarrow [root]$
- 7: **for** $k = 1$ to $length(Splits)$ **do**
- 8: $P_k \leftarrow Data\%[k]$
- 9: $h_t \leftarrow$ Extend the tree h_t from $current_node$ up to depth $Splits[k] \times max_depth$ using only $P_k\%$ of $window_size$
- 10: Add flags for the current leaf nodes to indicate that they correspond to exiting nodes
- 11: $current_nodes \leftarrow [current\ exiting\ nodes]$
- 12: **end for**
- 13: Calculate OOB Misclassification Rate for each class on D_{oob} using the last exit
- 14: Update sample weights w by increasing weights for wrongly classified samples
- 15: Append h_t to $Trees_list$
- 16: **end for**
- 17: **return** $Trees_list$

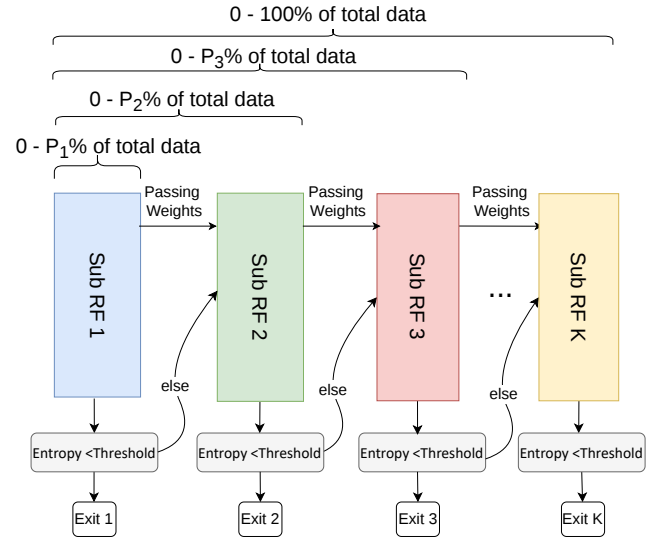


Fig. 3: Tree architecture in SEE-hRF

SEE-hRF Architecture and Training: RFs in SEE-hRF are divided into multiple sub-forests, each corresponding to an early exit, as shown in Figure 3. The first sub-forest includes a subset of trees that are used with the first data portion. Subsequently, the next sub-forest is invoked if the predictions from the first exit do not have a high confidence. Baseline RFs use the same weight for each tree in the forest when making classification decisions. However, this may provide lower accuracy for harder classification classes. To overcome

Algorithm 2 Training SEE-hRF

Require: Dataset D , data percentages $Data\%$, tree splits $Splits$, number of estimators $N_{\text{estimators}}$, maximum tree depth max_depth , total number of windows N

Ensure: List of trained trees with horizontal early-exits

- 1: Initialize sample weights $w(i) \leftarrow 1/N$ for all $i = 1, \dots, N$.
- 2: $Trees_list \leftarrow []$, $w_{\text{current}} \leftarrow w$
- 3: **for** $k = 1$ to $\text{length}(Data\%)$ **do**
- 4: $P_k \leftarrow Data\%[k]$, $T_k \leftarrow Splits[k]$
- 5: **for** $t = 1$ to T_k **do**
- 6: $D_{bag}, D_{oob} \leftarrow$ Weighted bootstrap sample and OOB from D
- 7: $h_{k,t} \leftarrow$ Train a decision tree using D_{bag} with features up to $P_k\%$ of window
- 8: Append $h_{k,t}$ to $Trees_list$
- 9: Compute OOB error $E^t(c)$ for each class c on D_{oob}
- 10: **for** each sample i in D_{oob} **do**
- 11: $c \leftarrow$ true class of i
- 12: **if** $y_i \neq h_{k,t}(\mathbf{x}_i)$ **then**
- 13: $w(i) \leftarrow w(i) \cdot (1 + E^t(c))$
- 14: **end if**
- 15: **end for**
- 16: Normalize w
- 17: **end for**
- 18: **end for**
- 19: **return** $Trees_list$

this, the core idea of sRF is to sequentially adapt the number of training samples per class for each tree and to employ a weighted majority voting during training in each sub-forest. Following the framework detailed in the previous section, each sub-forest uses the sequential sample re-weighting during training. In addition, it utilizes weighted majority voting mechanisms to improve classification accuracy.

Standard RFs use majority voting, implicitly assigning equal weight to each tree. In contrast, the proposed architecture modulates each tree's influence by its class-specific error rate. The weight of tree tr 's prediction for class c is:

$$L^{tr}(c) = 1 - E^{tr}(c) \quad (4)$$

The prediction score for a sample $\mathbf{x}$ to belong to class c is calculated by summing the weights of all trees tr that predict that class:

$$\text{Score}(c|\mathbf{x}) = \sum_{tr=1}^{N_T} \mathbb{I}(\text{Tree } tr \text{ predicts } c \text{ for } \mathbf{x}) \cdot L^{tr}(c) \quad (5)$$

where N_T is the total number of trees and $\mathbb{I}(\cdot)$ is the indicator function. The final class probability $P(c|\mathbf{x})$ is obtained by normalizing $\text{Score}(c|\mathbf{x})$ across all classes.

Weight Propagation Across Sub-forests: Overall, the SEE-hRF training process allows each sub-forest to achieve high accuracy by building a strong, complementary ensemble of trees. The training procedure, summarized in Algorithm 2, is repeated for each sub-forest to obtain the full RF with all exits. We note that the OOB samples in Algorithm 2 are obtained in the same manner as for SEE-vRF. Moreover, each subforest corresponding to an early exit k uses cumulative $P_k\%$ of the window data, as shown in line 7 of the algorithm. For example,

for a configuration with 3 exits, the values of P could be $P_1 = 30\%$, $P_2 = 60\%$, and $P_3 = 100\%$. In this case, exit two uses $0 \dots 60\%$ of data in a window. We also pass the sample weights across sub-forests to ensure that importance of samples learned in each sub-forest is passed to the next one. Specifically, the classification confidence or learned sample distribution from the final tree of sub-forest i is used to inform the initial weighting of samples for the first tree of sub-forest $i + 1$. The passing of information ensures that each sub-forest is connected to each other, and they are not trained in isolation.

Relationship of SEE-RFs to Boosted RFs: The proposed RF architectures depart from traditional RFs by using sequential re-weighting between trees. At the same time, while our architecture uses sequential re-weighting like boosting [36], it is different from boosted RFs because it uses full depth trees rather than shallow weak learners to reduce bias. The performance remains robust and overfitting is avoided since we use OOB samples to obtain weights. Independence is also preserved since the reweighting affects the importance given to labels while each tree is still trained independently.

C. Design Space Exploration for SEE-RFs

Design of both SEE-RF architectures involves the selection of number of early exits $K - 1$, the data portions t_i , split of trees either vertically or horizontally for each exit, and entropy thresholds for the confidence. The goal of this design space exploration (DSE) is to choose the above parameters such that both accuracy and energy savings are maximized. The straightforward method to select the parameters is to perform an exhaustive grid search with all possible combinations of the parameters and obtaining a Pareto frontier. Then, we can choose a design that offers the best trade-off for deployment on the devices. While exhaustive search can uncover promising architectures, it incurs high design time overhead since we must train RFs for each combination. Therefore, we randomly sample a subset of configurations from the exhaustive search space to search for promising SEE-RF configurations. We refer to this setup as *random grid search*.

Exhaustive search, including the *random grid search* may train unpromising candidate designs or include redundancy in designs, which leads to additional computational requirements. To overcome this, we utilize multi-objective Bayesian optimization (BO) [37] to perform efficient DSE. Key idea in BO is to learn surrogate models of each objective and then use an acquisition function to choose next design point such that the information gain about the true Pareto front is maximized. Unlike *random grid search*, which is restricted to a pre-specified coarse grid of discrete configurations, BO models the objective landscape with a surrogate and searches a continuous space, enabling it to identify better trade-offs between points that the grid never evaluates.

We define the objective functions for BO as follows:

$$\min [-\text{Acc}(\theta), E(\theta)] \quad (6)$$

where $\text{Acc}(\theta)$ is the model accuracy, $E(\theta)$ is the total energy consumption (including both sensor and compute energy), and $\mathcal{X}$ is the configuration space. We utilize the standard Gaussian

process (GP) surrogate models to choose designs for SEE-RFs. The BO is executed for a fixed number of iterations to obtain the Pareto optimal set. Each iteration uses the surrogate models and acquisition function to choose the next set of SEE-RF parameters for training and evaluation. Fixed iterations in the BO ensure that the training time is scalable for different datasets. BO leads to significant savings in the design time through the use of surrogate models, continuous embedding, and the acquisition function. The combination of these directs the search to promising regions, thus leading to faster search.

VI. EXPERIMENTAL EVALUATION

A. Experimental Setup

System Platform: We utilize the Raspberry Pi Zero 2W [22] to implement SEE-RFs. The Pi Zero’s single-core ARM Cortex A53 processor, 512 MB RAM, and absence of GPU or hardware accelerator closely mirror the constraints of wearable-class edge processors, where RF inference must rely solely on general-purpose comparison operations. Prior work has validated RF inference on comparable or more constrained embedded platforms [17, 18].

Datasets: We utilize six health and activity datasets for evaluating SEE-RFs.

Physical Activity Recognition (Shoaib) [4]: The Shoaib dataset provides accelerometer data for 10 users performing seven activities.

PAMAP2 [33]: PAMAP2, offers data from three accelerometers for five activities with nine users.

Epilepsy [38]: This dataset provides wrist acceleration data for six participants.

WESAD [39]: WESAD is a multi-modal dataset using wearable sensors for affect detection. It includes data from five sensors for 15 participants.

SelfRegulationSCP1 (SR-SCP1) [40]: The SR-SCP1 dataset includes Electroencephalography (EEG) data from six channels used in a control system driving spelling devices for paralyzed patients.

EMG Physical Action (EMG) [41]: The EMG dataset focuses on activity recognition for users who may exhibit aggression during tasks. It collects data using eight EMG sensors placed on the upper arms and legs.

SEE-RF Design Parameters: SEE-RFs use the following hyperparameters to balance accuracy and energy: number of trees and maximum depth, number of exits, tree splits for sub-forests or levels, data percentages, and the exit thresholds. Table I summarizes the possible values used for each hyperparameter in our DSE.

Evaluation Metrics: We utilize energy, accuracy, precision, recall, and F1-Score [42] as the evaluation metrics. As described in the previous sections, the primary goal of designing

SEE-RFs is to maximize sensor energy savings while maintaining accuracy. Memory footprint is used as an additional metric to evaluate the storage requirements for SEE-RFs.

Train and Test Splits: We use 60% data for training, 20% data for validation, and remaining 20% data for testing. All results reported in the following show the test accuracy.

Training Overhead: We train the SEE-RF models on a server with the Nvidia Titan T4 GPU. Random grid search for 900–1200 configurations takes about 3–18 hours while the BO for 15–45 configurations takes about one hour.

B. Baseline Approaches

To validate that SEE-RF’s energy savings are from its architectural design, we compare against baselines that systematically eliminate competing explanations. A conventional RF ensemble [19] trained on complete sensor windows serves as our primary baseline, establishing both the accuracy ceiling and maximum energy consumption reference against which all energy savings are measured. We additionally train a standalone gradient-boosted (GB) tree ensemble on complete sensor windows to test whether the choice of RF over other tree ensembles affects the conclusions. The GB baseline uses the same configuration as the RF classifiers within TEASER [21] and Non-Myopic Early Classifier (EC) [20] (50 estimators, max depth 30), for a fair comparison.

We also compare against three state-of-the-art early time-series classification methods that implement early exits or Adaptive RFs. Description of each approach is provided below.

TEASER [21]: This approach uses a two-tier architecture where early-stage classifiers at multiple temporal checkpoints assess classification confidence via one-class novelty detection. We configure it with checkpoints at 25%, 50%, and 75% of window duration to match SEE-RF’s evaluation points.

Non-Myopic EC [20]: The Non-Myopic EC [20] from the tslearn library [43] formulates early classification as a sequential decision problem balancing prediction accuracy against penalty on decision time. The approach aims to classify time-series data as early as possible in the data collection process. It includes a hyperparameter α that balances earliness and accuracy. We evaluate with $\alpha \in \{0.01, 0.05, 0.1, 0.2, 0.5\}$ and report results using the value that achieves the best harmonic mean of accuracy and earliness. We use a RF as the base classifier for consistency with the other baselines, so each exit point is an independent forest. One of the key limitations of the Non-Myopic EC classifier is that it trains a unique classifier for each possible time-series exit point. For instance, if an application has 200 data points in each window, the Non-Myopic approach trains 200 classifiers, which significantly increases the overhead. TEASER and Non-Myopic EC are also used as benchmarks for early time-series classification [44].

Adaptive RF [17]: The approach in [17] builds an ensemble of RF classifiers for deployment on low-power microcontrollers. At runtime, it adaptively chooses the number of trees to execute based on the latency/energy target or input complexity. Using a subset of trees introduces early exits in the Adaptive RF framework. However, we note that this approach does not

TABLE I: Hyperparameter ranges for DSE of SEE-RFs

Parameter	Range	Applicable To
Number of exits	2, 3, 4	SEE-RFs only
Number of estimators	4–90	All RF methods
Data percentages	25%–100%	SEE-RFs only
Tree depths	10–70	All RF methods
Tree splits	25%–100%	SEE-RFs only
Exit thresholds	$H_{\min}$ – $H_{\max}$	SEE-RF only

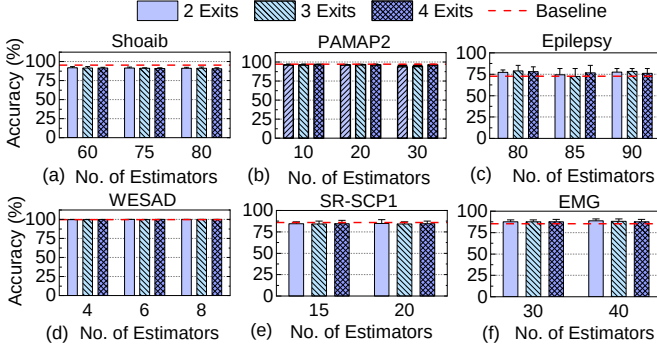


Fig. 4: SEE-vRF: Accuracy vs. No. of Estimators

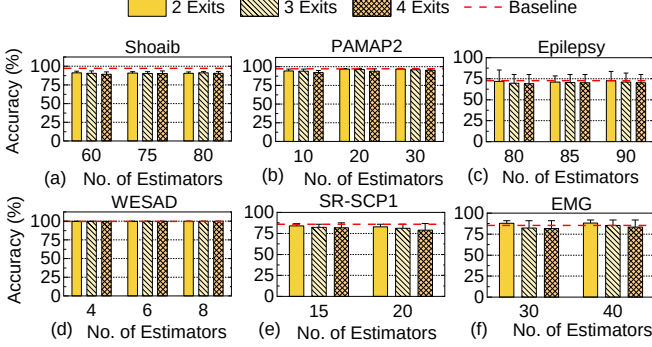


Fig. 5: SEE-hRF: Accuracy vs. No. of Estimators

utilize partial data windows. We report energy savings from computation for the Adaptive RF approach.

Overall, neither of the three methods physically controls sensor power states; we map their prediction time to the fraction of the sensor window consumed to estimate energy savings, which represents an optimistic upper bound.

C. SEE-RF Performance with Number of Trees

We start the experimental results with an evaluation of the accuracy and energy of both SEE-RF versions as we vary the number of estimators in each RF. The number of estimators is an important metric since it directly impacts the overall size of the RFs and runtime overhead.

Figures 4 and 5 show the classification accuracy for random grid search DSE with different numbers of exits and estimators for all datasets. Each bar shows the average accuracy with a given combination of estimators and exits. The bars also show the maximum accuracy achieved for the combination of exits and estimators since not all configurations result in high accuracy. We see that the maximum accuracy for all datasets is close to the baseline RF accuracy that uses 100% of data and no early exits. Comparing the two SEE architectures, we see comparable performance across most datasets. SEE-hRF provides higher average accuracy in some datasets compared to SEE-vRF. This advantage stems from the fact that horizontal exits aggregate decisions from full-depth trees, preserving the representational power of the ensemble. In contrast, vertical exits may lead to lower accuracy when early exits occur at shallow depths with fewer nodes (part of the tree levels), especially on complex datasets. At the same

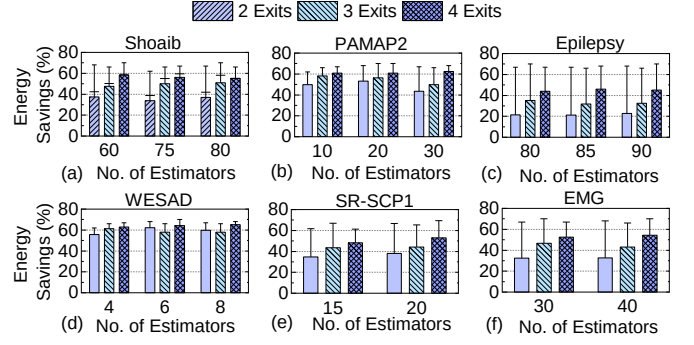


Fig. 6: SEE-vRF: Energy vs. No. of Estimators

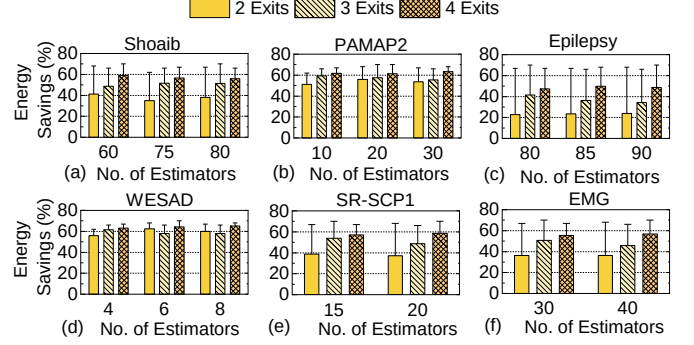


Fig. 7: SEE-hRF: Energy vs. No. of Estimators

time, SEE-vRFs typically incur fewer computations compared SEE-hRFs, making them more efficient computationally.

Next, Figures 6 and 7 show the sensing energy savings with change in the number of exits and number of estimators for both SEE-RF versions. The energy savings are measured compared to the default case where sensors are active for the entire window duration. The energy savings are computed with simulations by measuring the number of windows that exit at early exits and using the corresponding latency to compute energy. We note the energy savings values include the overhead of switching sensors on and off. Actual sensor measurements are shown in Section VI-H1. The figures show that both RF versions achieve 40–60% simulated sensing energy savings across datasets. The energy savings increase as we increase the number of exits. This is because having additional exits provides better opportunities for early classification as soon as sufficient confidence is achieved.

Figures 4 and 5 do not show significant changes in accuracy as they represent overall average accuracy when using different exit combinations. This means that most configurations achieve similar accuracy by appropriately using different exits. The standard deviation whiskers show the variation seen in the accuracy values. While the accuracy variation is lower, we see significant variation in energy savings in Figures 6 and 7. This is because configurations that use later exits may have good accuracy at the cost of low energy savings. Overall, these results show the potential of SEE-RFs in achieving significant sensing energy savings without affecting the accuracy.

D. Analysis of Exit Distribution

One of our key goals is to enable early exits to enable energy savings. Specifically, utilizing early exits enables us

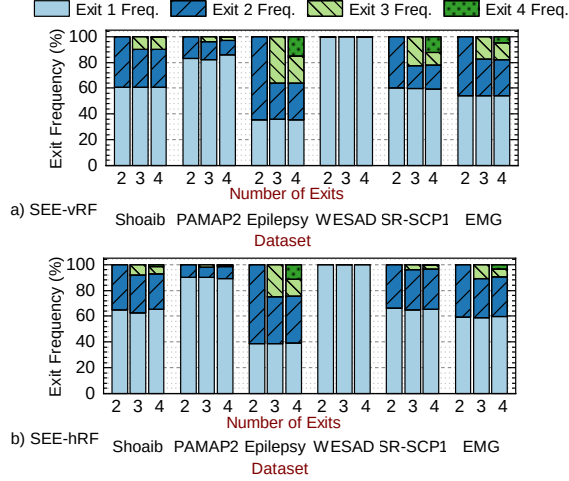


Fig. 8: Exit frequency taken across datasets for a) SEE-vRF and b) SEE-hRF

to transition sensors into low power mode for energy savings. To this end, this section analyzes the distribution of exits taken across all datasets for the two SEE-RF architectures. Figure 8 shows the distribution of average exit frequencies as a function of the number of exits and datasets. Each bar represents the average exit frequency across all possible configurations evaluated as part of DSE. We see that at least 40% windows exit at the first exit for all datasets in both SEE-RF versions. Exit two shows the next highest frequency, while exits three and four are rarely used. Moreover, WESAD exits at the first exit almost 100% of the time, thus showing that partial window data is sufficient to make confident inference. Overall, these results demonstrate that both SEE-RF versions consistently enable earlier exits for high energy savings.

E. Comparison of BO and Random Grid Search for DSE

We introduce BO to uncover promising SEE-RF designs with minimal training configurations. To this end, this section compares the accuracy-energy savings trade-off achieved by the two DSE methods. Figures 9 and 10 show the Pareto front designs from both DSE methods. We also show the baseline RF with no early exits with a blue triangle. We see that for both SEE-RFs BO and random grid search achieve Pareto fronts that are close to each other. Even when they are not close for some datasets, such as Epilepsy, BO achieves designs that dominate those from random grid search.

This shows that leveraging BO for DSE is promising to obtain highly efficient designs while training and evaluating significantly fewer designs. As shown in the training overhead above, BO achieves comparable or superior Pareto fronts with a 20–80 $\times$ reduction in the number of evaluated configurations.

F. Comparison with Baseline Approaches

This section compares the performance of proposed RF architectures against the baseline approaches. We start with the comparison of the performance with RFs and GB classifiers in Table III. The table compares the accuracy obtained by RF and GB classifiers with full data. The GB baseline consistently underperforms the Baseline RF across all six datasets, with particularly large gaps on Shoaib (76.98% vs. 95.00%) and

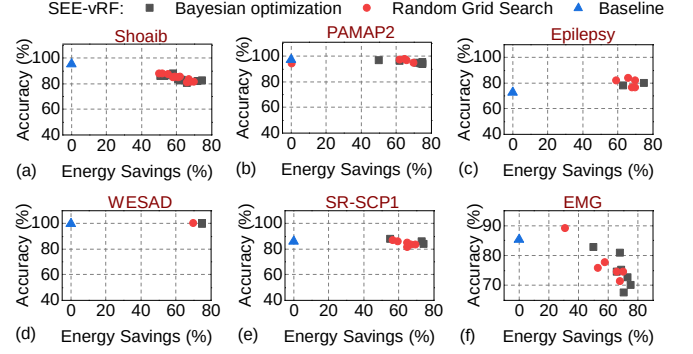


Fig. 9: SEE-vRF Pareto optimal points for two DSE methods

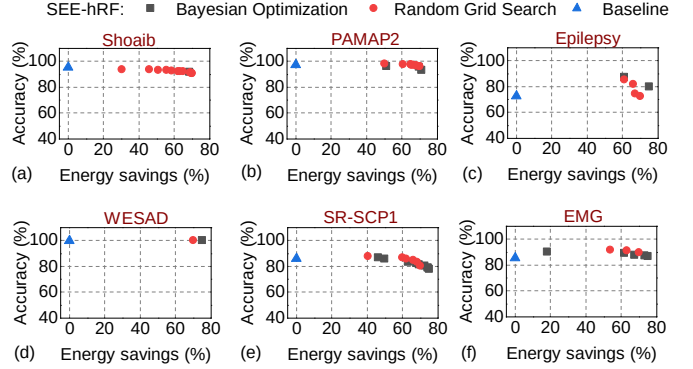


Fig. 10: SEE-hRF Pareto optimal points for two DSE methods

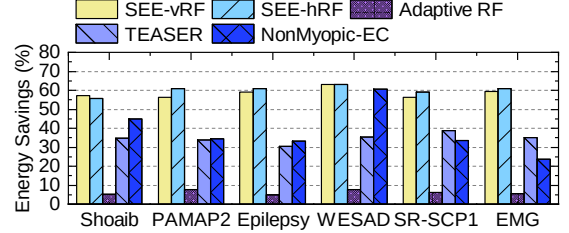


Fig. 11: Comparison of energy savings across all approaches

EMGPhysical (74.76% vs. 90.42%). This validates the choice of RF as the base ensemble for SEE-RF: under identical hyperparameters, RFs extract substantially more discriminative information from time-series windows than GB trees.

Next, Table II reports accuracy, precision, recall and F1-score across all datasets for the three state-of-the-art approaches and SEE-RFs. The tables report the mean values followed by the standard deviation for all metrics in parenthesis. We run the state-of-the-art approaches with multiple seeds to capture average performance, while the SEE-RF data is reported for the Pareto-optimal front of configurations. The results show that both SEE-RFs maintain strong performance across all metrics. We also see that baseline approaches typically have lower accuracy and other metrics, except for Shoaib and PAMAP2 datasets. The baselines approaches achieve accuracies comparable or slightly better than SEE-RFs for these datasets. This comes at significantly lower energy savings, as we show next. Moreover, the Non-Myopic EC approach has higher overhead due to training one classifier for each data point, which makes it unsuitable for low-memory devices.

Finally, Figure 11 shows the comparison of energy savings across all datasets and approaches. The energy savings are

TABLE II: Performance comparison across datasets for SEE-RFs and baseline approaches.

Metric	Method	Shoaib	PAMAP2	Epilepsy	WESAD	SCP	EMG
Accuracy(%)	Adaptive RF [17]	94.9 (0.5)%	96.4 (0.7)%	80.4 (3.6)%	96.6 (0.2)%	82.1 (2.2)%	87.5 (1.7)%
	TEASER [21]	96.1 (0.4)%	96.5 (0.4)%	80.2 (4.6)%	92.5 (0.3)%	82.5 (2.1)%	87.7 (2.5)%
	Non-Myopic EC [20]	90.1 (2.0)%	96.2 (1.0)%	66.5 (5.2)%	95.3 (0.5)%	80.8 (1.9)%	84.5 (3.2)%
	SEE-vRF	92.3 (0.8)%	96.7 (0.4)%	77.6 (2.6)%	99.97 (0.0)%	82.1 (2.5)%	90.2 (1.7)%
	SEE-hRF	92.13 (0.77)%	96.74 (0.49)%	80.14 (1.89)%	100.0 (0.0)%	84.18 (2.90)%	92.68 (0.68)%
Precision	Adaptive RF [17]	0.962 (0.004)	0.966 (0.004)	0.810 (0.043)	0.966 (0.003)	0.835 (0.021)	0.889 (0.023)
	TEASER [21]	0.962 (0.004)	0.966 (0.004)	0.810 (0.043)	0.925 (0.003)	0.835 (0.021)	0.889 (0.023)
	Non-Myopic EC [20]	0.901 (0.021)	0.963 (0.010)	0.688 (0.073)	0.953 (0.005)	0.821 (0.022)	0.849 (0.033)
	SEE-vRF	0.928 (0.007)	0.968 (0.004)	0.776 (0.024)	0.9997 (0.000)	0.823 (0.026)	0.908 (0.016)
	SEE-hRF	0.924 (0.008)	0.968 (0.005)	0.812 (0.012)	1.000 (0.000)	0.848 (0.029)	0.930 (0.008)
Recall	Adaptive RF [17]	0.949 (0.005)	0.964 (0.007)	0.804 (0.036)	0.966 (0.002)	0.821 (0.022)	0.875 (0.017)
	TEASER [21]	0.961 (0.004)	0.965 (0.004)	0.802 (0.046)	0.925 (0.003)	0.825 (0.021)	0.877 (0.025)
	Non-Myopic EC [20]	0.901 (0.020)	0.962 (0.010)	0.665 (0.052)	0.966 (0.002)	0.808 (0.019)	0.845 (0.032)
	SEE-vRF	0.923 (0.008)	0.967 (0.004)	0.776 (0.026)	0.9997 (0.000)	0.821 (0.025)	0.902 (0.017)
	SEE-hRF	0.921 (0.008)	0.967 (0.005)	0.801 (0.019)	1.000 (0.000)	0.842 (0.029)	0.927 (0.007)
F1-score	Adaptive RF [17]	0.948 (0.005)	0.964 (0.007)	0.804 (0.037)	0.966 (0.002)	0.820 (0.023)	0.876 (0.017)
	TEASER [21]	0.961 (0.004)	0.965 (0.004)	0.801 (0.045)	0.925 (0.003)	0.823 (0.022)	0.878 (0.025)
	Non-Myopic EC [20]	0.899 (0.021)	0.962 (0.010)	0.662 (0.054)	0.953 (0.005)	0.806 (0.018)	0.845 (0.032)
	SEE-vRF	0.924 (0.008)	0.967 (0.004)	0.770 (0.026)	0.9997 (0.000)	0.821 (0.025)	0.903 (0.017)
	SEE-hRF	0.922 (0.008)	0.967 (0.005)	0.795 (0.023)	1.000 (0.000)	0.842 (0.029)	0.927 (0.007)

TABLE III: Full-data baseline accuracy (%). Both models use 50 estimators, max depth 30, trained on complete sensor windows with no early exits

Dataset	Baseline RF (%)	GB (%)
Epilepsy	74.54	56.36
PAMAP2	97.80	93.06
Shoaib	95.00	76.98
WESADchest	100.00	99.97
EMGPhysical	90.42	74.76
SelfRegulationSCP1	84.89	75.56

obtained through simulations for all methods. The energy savings for SEE-RFs are obtained from the Pareto front design configurations. We see that SEE-RF methods consistently have higher energy savings than all the baseline approaches. Non-Myopic EC has the highest energy savings among the baselines, in particular, its energy savings are close to SEE-RFs for the WESAD dataset. However, it comes at the cost of lower accuracy and larger memory overhead from training a classifier for each data point. Non-Myopic EC also exhibits high run-to-run variance in energy savings, since its single-timestep stopping rule is sensitive to the early-window classifier, whereas SEE-RF's entropy routing yields more stable savings. Adaptive RFs have low energy savings since they do not explicitly turn off the sensors, instead full data with early exits are used. TEASER achieves 30–40% energy savings, which is consistently lower than SEE-RFs. In summary, these results show that SEE-RFs obtain the best trade-off in terms of accuracy and energy savings.

G. Ablation Studies

This section performs multiple ablation studies to better understand results of SEE-RFs.

Importance of Sensor Awareness: One of the key contributions of the proposed approach is being sensor-aware by exiting early whenever sufficient data are available. To evaluate the energy savings of sensor-awareness, we train two versions of the proposed SEE-RFs. The first version includes early exits while providing 100% of sensor data at the beginning. More specifically, it uses the entropy-based confidence logic

TABLE IV: Impact of sensor awareness: progressive vs. full data collection (accuracy, energy, ECE)

Model	WESAD			Shoaib		
	Acc%	E%	ECE	Acc%	E%	ECE
SEE-vRF (100%)	100	0.0	0.0	92.38	0.0	0.30
SEE-vRF (aware)	100	75.0	0.0	93.17	58.0	0.32

TABLE V: Effect of sensor awareness: comparison of sequential RF with horizontal exits using 100% of data at each exit versus sensor-aware variant (accuracy, energy savings and ECE)

Model	WESAD			Shoaib		
	Acc%	E%	ECE	Acc%	E%	ECE
SEE-hRF (100%)	100	0.0	0.0	90.63	0.0	0.05
SEE-hRF (aware)	99.93	70.0	0.0	90.48	46.21	0.04

for exits and can use data from any time step at any node of the tree. In contrast, the second version uses sensor-aware exits where increasing data percentages are used at each exit. Tables IV and V show that both sensor-aware SEE-RF versions show significant energy savings while maintaining comparable accuracy. The results also show low ECE scores for both SEE-vRF and SEE-hRF on WESAD, which indicates that the models are well calibrated to WESAD. On Shoaib, SEE-hRF exhibits low ECE value, whereas SEE-vRF exhibits a high ECE. This can be explained by a considerable gap between the model confidence (the maximum class probability) and accuracy in SEE-vRF. Specifically, in SEE-vRF, the early exit predictions come from a set of shallower trees that traverse fewer decision splits than those in SEE-hRF, leaving the classification probability low despite achieving good accuracy. For instance, around 70% of the windows in this configuration had an average maximum probability of 0.65 with an accuracy of 98%. The large accuracy–confidence gap increases ECE values, as defined in Equation 2.

Importance of Entropy Thresholds: The entropy thresholds at each exit play an important role in energy savings and accuracy. Higher values of entropy threshold lead to higher likelihood of early exit, albeit with lower confidence. As such, it may lead to lower accuracy. To evaluate the effect

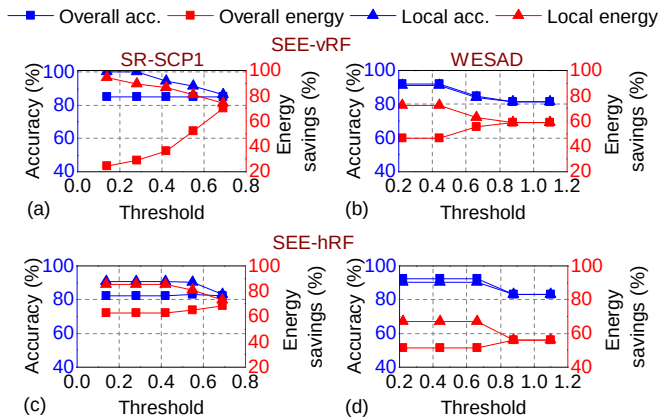


Fig. 12: Change in accuracy and energy savings with entropy threshold at exit 1 for two datasets. The left y-axis in blue corresponds to accuracy shown in blue. The right y-axis in red corresponds to the energy savings shown in red.

TABLE VI: Static Truncation RF: Accuracy (%) when training and testing on the first $P\%$ of each sensor window.

Data	Accuracy (%)					
(%)	Shoaib	PAMAP2	Epilepsy	WESAD	SCP	EMG
10	85.53	96.05	73.94	99.99	69.18	87.33
20	86.98	95.83	73.94	100.0	80.29	87.11
30	86.96	96.83	77.58	100.0	84.00	89.56
40	88.54	97.72	75.15	100.0	85.93	88.39
50	89.87	97.80	81.82	100.0	85.92	88.50
100	95.00	97.80	74.54	100.0	84.89	90.42

of thresholds, we train the RFs and vary the exit threshold at the first early exit, while fixing thresholds for other exits.

Figure 12 shows the change in accuracy and energy savings with change in threshold. The left y-axis in each figure represents the accuracy in blue color while the right y-axis represents the energy savings with red color. The accuracy plots show the overall accuracy across all exits and the local accuracy at the first exit. As expected, we see that energy savings increase as we increase the threshold values, while the accuracy typically decreases. We must choose the appropriate trade-off for accuracy and energy in these cases to balance both metrics. We also observe that the overall accuracy remains constant as we increase the threshold in some cases. Overall, this study shows that we must choose the highest threshold that provides the required classification accuracy levels.

Static Truncation RFs: A simple but naive approach is to collect less sensor data and train distinct standard RFs on the shorter windows, avoiding architectural complexity. To address this directly, we train standard RFs on temporally truncated windows at fixed percentages (10%, 20%, 30%, 40%, 50%, 100%) of the full window duration, cutting all sensor channels at the same temporal point to simulate a fixed sensor duty-cycle. A separate RF is trained at each truncation level. Energy savings are approximately $(1 - P/100) \times 100\%$. The static truncation commits every sample to the same fixed cutoff regardless of classification difficulty.

The static truncation results in Table VI expose a nuanced picture. On datasets where the discriminative signal appears early in the window, such as PAMAP2 and WESADchest, even 10% of data suffices for near-ceiling accuracy (96.05% and 99.99%, respectively). On these datasets, SEE-RF’s advantage

TABLE VII: Data fractions allocated at each exit.

Dataset	SEE-vRF				SEE-hRF		
	E1	E2	E3	E4	E1	E2	E3
Shoaib	0-47%	0-57%	0-69%	0-100%	0-25%	0-100%	-
PAMAP2	0-38%	0-53%	0-100%	-	0-25%	0-100%	-
Epilepsy	0-33%	0-100%	-	-	0-25%	0-37%	0-100%
WESAD	0-25%	0-100%	-	-	0-44%	0-62%	0-100%
SCP	0-31%	0-100%	-	-	0-30%	0-44%	0-100%
EMG	0-38%	0-56%	0-100%	-	0-33%	0-100%	-

is not raw accuracy but its adaptive per-sample routing: static truncation commits every sample to the same fixed cutoff, while SEE-RF allows individually hard samples to collect more data. In contrast, datasets like Shoaib and SelfRegulationSCP1 show substantial accuracy degradation under truncation: Shoaib drops from 95.00% at full data to 85.53% at 10%, and SelfRegulationSCP1 drops from 84.89% to 69.18%, demonstrating that these tasks genuinely require later temporal features for a subset of samples.

Effects of Sequential Re-Weighting on SEE-RFs: The proposed SEE-RF architectures use sequential re-weighting during training as a function of performance on OOB samples. This procedure departs from traditional RFs that do not use re-weighting. To this end, we perform an ablation study where we train the two SEE-RF architectures without sequential re-weighting. We observe that both approaches offer comparable trade-off in accuracy and energy savings, with sequential re-weighting obtaining higher accuracy and energy savings for some datasets. For instance, SEE-hRF with re-weighting for the WESAD dataset obtains average accuracy and energy savings of 99% and 62%, respectively, when compared to 89% and 49% for standard RF training. This shows that sequential re-weighting provides information about important labels for later exits and helps in obtaining better performance. At the same time, we note that the proposed SEE-RF architectures are compatible with standard RF training.

H. Implementation Overhead and Deployment

This section discusses the implementation overhead and real-world deployment considerations for SEE-RF. We first describe the overhead of executing the early exit models on the Raspberry Pi Zero 2W. The training process for both SEE-RF versions yield a set of early exit configurations with corresponding energy savings and accuracy. Promising configurations that offer Pareto-optimality are chosen from the set of trained configurations for potential deployment on embedded devices. We choose a configuration that balances accuracy and energy savings. Specifically, we save one representative Pareto-optimal model with two to four exits for each SEE-RF version and execute it on the device using Python for each dataset. Table VII shows the number of exits and percentage of data used for each exit in both RF versions for all datasets. We see that SEE-vRF has all possible number of exits ranging from two to four, while SEE-hRF has an even split between two and three exit versions. The first exit uses 25–40% data in most datasets across both RF versions, thus representing a potential for large energy savings. Subsequent exits in all RFs use additional data, that is, each exit uses cumulative data available up to the exit point.

TABLE VIII: Energy Comparison Between SEE-vRF and SEE-hRF

Dataset	SEE-vRF		SEE-hRF	
	Time (s)	Energy (mJ)	Time (s)	Energy (mJ)
Shoaib	0.92	710.50	0.18	334.94
PAMAP2	0.10	127.68	0.07	130.92
Epilepsy	0.35	493.12	0.19	361.04
WESAD	0.02	42.27	0.02	34.96
SCP	0.10	213.55	0.05	104.36
EMG	0.26	522	0.10	361.04

TABLE IX: Summary of memory overhead of the RF classifiers

Dataset	Memory Storage (KB)		
	Baseline RF	SEE-vRF	SEE-hRF
Shoaib	345	359	299
PAMAP2	33	48	36
Epilepsy	70	74	62
WESAD	0.3	19	10
SCP	15	16	11
EMG	57	65	52

Table VIII shows the average execution time and energy for performing a single inference for each dataset using both RF methods. We see that SEE-hRF executes in under 200 ms and consumes under 400 mJ across all datasets. SEE-vRF execution time and energy vary more by dataset, reaching up to 0.92 s and 710.50 mJ on Shoaib, reflecting its deeper per-tree traversal. In both cases, the early-exit entropy computation adds negligible overhead relative to the core RF inference.

We also compare the memory requirements of SEE-RFs when compared to the baseline RFs with no early exits. Our key goal here is to obtain RFs that have similar memory footprint to that of the baseline RFs while achieving energy savings. Table IX shows the memory requirements of the SEE-RFs along with the baseline RF size for all datasets. The memory requirements are estimated by calculating the number of nodes in the RFs and the data requirements of each node. We see that the memory requirements for the SEE-RFs are similar to those of the baseline RFs or slightly larger for five of six datasets. The exception is WESAD, whose baseline RF is unusually compact (0.3 KB) due to the small feature space and high class separability, making the relative increase to 10–19 KB appear large. However, all SEE-RF models remain under 360 KB in absolute terms, well within the memory budget of typical edge devices. Overall, SEE-RFs achieve significant energy savings with negligible memory overhead in practice.

1) *Real-World Deployment of SEE-RFs*: The energy savings estimated in previous sections rely on percentage of data used for making decisions. This is because we do not use the respective sensors to perform measurements of energy consumption. While this is useful, it does not consider the actual real-world issues of switching sensor status and integration of multiple sensors. To this end, this section integrates two motions sensors with the Raspberry Pi Zero 2W device to implement SEE-RFs, as shown in Figure 13. Specifically, we first save the sensor status information for the Shoaib dataset. Then, we use Raspberry Pi device to change the sensor status for each window following the trace. Table X shows the sensing energy savings compared to the default case of using all sensor data. Power measurements are done using an external INA219 sensor [45]. The table also shows

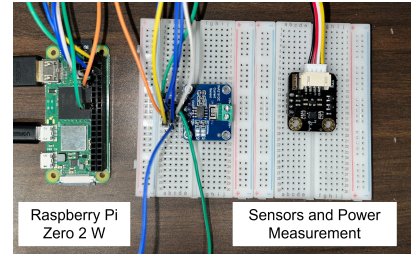


Fig. 13: Raspberry Pi Zero 2W setup

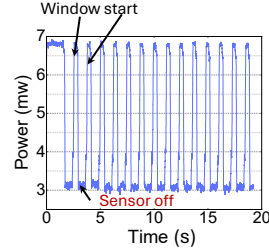


Fig. 14: Epilepsy power profile

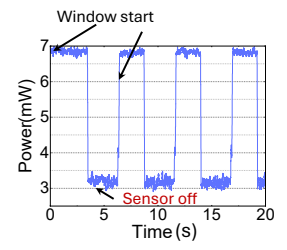


Fig. 15: Scenario 2 power profile

a comparative analysis where we consider different scenarios of time-series data exiting early. For example, the second row shows that 40% of data exit at Exit 1, 20% at Exit 2 and so on. We see that SEE-RFs save considerable energy in all scenarios, thus showing the benefits of the proposed approach.

Next, Figures 14 and 15 show the behavior of sensor power consumption as we use early exits in windows for the epilepsy dataset and scenario 2 in Table X. We see that there is a significant variation in the sensor power consumption between active and sleep states. Moreover, the switch between states is almost instantaneous without significant energy consumption. This further enforces the fact that early exits with partial window data can enable energy savings for wearable applications.

I. SEE-vRF and SEE-hRF Trade-off

SEE-hRF and SEE-vRF are complementary architectures and either can be used in time-series applications.

The key difference between the two is that SEE-vRF splits a given tree in an RF into multiple levels to introduce intermediate exits. In contrast, SEE-hRF uses a subset of full-depth trees for early exits. The shallow nature of early exits in SEE-vRF can lead to lower confidence in some datasets, such as Shoaib. Overall, they show comparable accuracies with SEE-hRF showing higher accuracies in some cases. They also use similar data percentages at exits, as shown in Table VII.

Comparing the accuracies between SEE-hRF and SEE-vRF, we see that SEE-hRF achieves higher accuracies in some cases. This is because SEE-vRF aggregates all trees predictions at the taken exit enabling early exits with fewer node evaluations, hence it uses fewer parameters for a given depth. The predictions are made with a subset of tree levels (fewer total nodes) at early exits. Therefore, it can lead to lower accuracy compared to SEE-hRF. In contrast, SEE-hRF uses the full depth and breadth of sub-forests at all exits, leading to higher accuracy. Our goal is to obtain trees with roughly similar memory requirements, however, adding more depth to SEE-vRF can lead to higher accuracy.

Next, both RF architectures employ comparisons at each node, thus the computations at each node are similar. The

TABLE X: Energy savings on Raspberry Pi

Experiment	Energy Savings (%)
SEE-RF Trace	42
Scenario 1	37
Scenario 2	29

computational complexity depends on the dataset and associated size of the RFs. At the same time, for a given tree, SEE-vRF typically incurs fewer computations since it splits a tree into multiple sections to enable early exits. We also see that SEE-vRF has higher memory requirements than SEE-hRF. This is typically to ensure that the trees have sufficient levels to provide predictions at early exits. In particular, SEE-hRF is beneficial in low-memory devices since we can load one tree at a time when performing inference. Later exits do not need RF parameters from prior exits. As such, the memory requirements at any given point of time are lower compared to SEE-vRF. At the same time, SEE-vRF reduces the memory transfer overhead since a single set of parameters is sufficient to evaluate all exits for a given data window.

Finally, both SEE-RF versions show comparable energy consumption and execution time for a single inference, except for Shoaib dataset when using SEE-vRF. The Shoaib dataset has a higher latency and energy due to its deeper tree traversal. We also note that SEE-hRF is able to utilize standard optimized libraries, such as scikit-learn [46], to implement the trees, thus achieving better latency. On the other hand, SEE-vRF uses custom implementation to insert exits at intermediate tree levels, thus leaving room for further optimizations.

In summary, both SEE-RF versions provide comparable performance. Users can choose either version based on their dataset and memory needs. SEE-hRF is suitable for low-memory devices since only one tree needs to be loaded at a time for inference. In contrast, SEE-vRF is suitable to avoid reloading of different tree parameters to infer multiple exits.

J. Source Code and Reproducibility

We release the source code, datasets, and model architectures on Github as an open-source release at: <https://github.com/gmbhat/see-rf>. The repository contains all necessary source code to train SEE-RF models with the datasets used in this paper. The repository also provides pre-trained models for use with the datasets. Finally, the repository includes scripts to measure power consumption using INA219 sensors on the Raspberry Pi Zero 2W board.

VII. CONCLUSION

Several real-world applications require time-series processing for identification of relevant parameters. ML models used for time-series applications typically use data from complete segments for classification. This can lead to high sensing energy consumption for the device. We proposed SEE-RF classifiers to utilize partial data in making classification decisions. The key idea in the proposed approach is to utilize partial data in a segment to perform classification for easier classes. Experimental results with six diverse datasets showed that the proposed approach achieves 40–60% simulated sensing

energy savings on average. Moreover, real-world experiments on a Raspberry Pi Zero 2W with two sensors showed that the proposed approach can provide 29–42% energy savings depending on the scenario. Although we focus on RFs due to their well-calibrated confidence estimates, low MCU overhead, and natural mapping between tree depth and progressive data, the proposed early-exit architectures proposed are applicable to other tree ensembles as well. Our immediate future work will combine the sensor selection problem with early exits to enable further energy savings. We will also aim to deploy SEE-RFs in real-world health applications.

ACKNOWLEDGMENT

The authors confirm that generative AI was not used to create scientific ideas, text, figures, or technical conclusions. Google Gemini was used to clean up BibTeX entries and format them correctly.

REFERENCES

- [1] F. J. Dian, R. Vahidnia, and A. Rahmati, "Wearables and the Internet of Things (IoT), Applications, Opportunities, and Challenges: A Survey," *IEEE Access*, vol. 8, pp. 69 200–69 211, 2020.
- [2] X. Liu and N. Ansari, "Toward Green IoT: Energy Solutions and Key Challenges," *IEEE Commun. Mag.*, vol. 57, no. 3, pp. 104–110, 2019.
- [3] A. J. Espay *et al.*, "Technology in Parkinson's Disease: Challenges and Opportunities," *Movement Disorders*, vol. 31, no. 9, pp. 1272–1282, 2016.
- [4] M. Shoaib *et al.*, "Fusion of Smartphone Motion Sensors for Physical Activity Recognition," *Sensors*, vol. 14, no. 6, pp. 10 146–10 176, 2014.
- [5] D. Hussein and G. Bhat, "CIM: A Novel Clustering-Based Energy-Efficient Data Imputation Method for Human Activity Recognition," *ACM Trans. Embedd. Comput. Syst.*, vol. 22, no. 5s, pp. 1–26, 2023.
- [6] A. Kansal *et al.*, "Power Management in Energy Harvesting Sensor Networks," *ACM Trans. Embedd. Comput. Syst.*, vol. 6, no. 4, p. 32, 2007.
- [7] Y. Tuncel *et al.*, "Towards Wearable Piezoelectric Energy harvesting: Modeling and Experimental Validation," in *Proc. Int. Symp. on Low Power Electron. and Des.*, 2020, pp. 55–60.
- [8] N. Yamin and G. Bhat, "Indoor-Outdoor Energy Management for Wearable IoT Devices with Conformal Prediction and Rollout," *IEEE Trans. on Comput.-Aided Des. of Integr. Circuits and Syst. (TCAD)*, vol. 43, no. 11, pp. 3370–3381, 2024.
- [9] D. Hussein, N. Yamin, and G. Bhat, "EMI: Energy Management meets Imputation in Wearable IoT Devices," *IEEE Trans. on Comput.-Aided Des. of Integr. Circuits and Syst. (TCAD)*, vol. 43, no. 11, pp. 3792–3803, 2024.
- [10] World Health Organization, "Global Report on Diabetes," 2016. [Online]. Available: <https://www.who.int/publications/i/item/9789241565257>
- [11] —, "Hypertension Fact Sheet," 2023. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/hypertension>
- [12] G. Bhat *et al.*, "REAP: Runtime Energy-Accuracy Optimization for Energy Harvesting IoT Devices," in *Proc. Des. Autom. Conf.*, 2019, pp. 1–6.
- [13] M. R. Nakhkash *et al.*, "Analysis of Performance and Energy Consumption of Wearable Devices and Mobile Gateways in IoT Applications," in *Proc. Int. Conf. Omni-Layer Intell. Syst.*, 2019, pp. 68–73.
- [14] A. Ozanne *et al.*, "Wearables in Epilepsy and Parkinson's disease – A Focus Group Study," *Acta Neurologica Scandinavica*, vol. 137, no. 2, pp. 188–194, 2018.
- [15] D. Hussein, C. Ugwu, G. Bhat, and J. R. Doppa, "Uncertainty-Aware Energy Management for Wearable IoT Devices with Conformal Prediction," in *2025 62nd ACM/IEEE Des. Autom. Conf.*, 2025, pp. 1–7.
- [16] D. Hussein, C. Ugwu, G. Bhat, and J. Doppa, "Trading-Off Performance and Sustainability in Internet of Things: An Uncertainty-Aware Hierarchical Energy Management Approach," *ACM Trans. Des. Autom. Electron. Syst.*, Jun. 2026, just Accepted.
- [17] F. Daghero *et al.*, "Dynamic Decision Tree Ensembles for Energy-Efficient Inference on IoT Edge Nodes," *IEEE Internet of Things J.*, vol. 11, no. 1, pp. 742–757, 2023.

- [18] M. Wainakh, M. Magno, and L. Benini, "TinyTree: Energy-efficient Tree-based Ensemble Models for Microcontroller-class Hardware," *IEEE Internet of Things J.*, vol. 10, no. 14, pp. 12 573–12 584, 2023.
- [19] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [20] A. Dachraoui, A. Bondu, and A. Cornuejols, "Early Classification of Time Series as a Non Myopic Sequential Decision Making Problem," in *Proc. of the Joint Euro. Conf. on Machine Learn. and Knowledge Discovery in Databases*, 2015, pp. 433–447.
- [21] P. Schäfer and U. Leser, "TEASER: Early and accurate time series classification," *Data Mining and Knowledge Discovery*, vol. 34, no. 5, pp. 1336–1362, 2020.
- [22] Raspberry Pi Ltd, "Raspberry Pi Zero 2 W," <https://www.raspberrypi.com/products/raspberry-pi-zero-2-w/>, 2021, accessed: 2025.
- [23] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks," in *Proc. 23rd Int. Conf. on Pattern Recognition (ICPR)*, 2016, pp. 2464–2469.
- [24] G. Huang *et al.*, "Multi-Scale Dense Networks for Resource Efficient Image Classification," in *Int. Conf. on Learning Representations*, 2018.
- [25] S. Liu *et al.*, "AdaDeep: A Usage-Driven, Automated Deep Model Compression Framework for Enabling Ubiquitous Intelligent Mobiles," *IEEE Trans. on Mobile Comput.*, vol. 20, no. 10, pp. 3282–3297, 2020.
- [26] B. Fang, X. Zeng, and M. Zhang, "NestDNN: Resource-Aware Multi-Tenant On-Device Deep Learning for Continuous Mobile Vision," in *Proc. ACM MobiCom*, 2018, pp. 115–127.
- [27] B. Srinovski *et al.*, "An Energy Aware Adaptive Sampling Algorithm for WSN with Power Hungry Sensors and Energy Harvesting," *Sensors*, vol. 16, no. 4, p. 448, 2016.
- [28] X. Xu *et al.*, "Adaptive Sampling-Based Information Collection for Wearable Sensors in WBAN," *Sensors*, vol. 16, no. 9, p. 1385, 2016.
- [29] K. Trapeznikov and V. Saligrama, "Supervised Sequential Classification Under Budget Constraints," in *Proc. 16th Int. Conf. on Artificial Intell. and Stat.*, vol. 31, 2013, pp. 581–589.
- [30] P. Zappi *et al.*, "Activity Recognition from On-Body Sensors: Accuracy-Power Trade-Off by Dynamic Sensor Selection," in *Proc. European Conf. on Wireless Sensor Netw.*, vol. 4913, 2008, pp. 17–33.
- [31] S. Peter *et al.*, "Cost Efficient Gradient Boosting," in *Advances in Neural Info. Proc. Syst.*, vol. 30, 2017.
- [32] D. Hussein *et al.*, "Sensor-Aware Early-Exit Neural Networks for Energy-Efficient Time-Series Classification on IoT Devices," *arXiv preprint arXiv:2407.08715*, 2024.
- [33] A. Reiss and D. Stricker, "Introducing a New Benchmarked Dataset for Activity Monitoring," in *Int. Symp. Wearable Comput.*, 2012, pp. 108–109.
- [34] C. Guo *et al.*, "On Calibration of Modern Neural Networks," in *Int. Conf. on Machine Learn.*, 2017, pp. 1321–1330.
- [35] M. P. Naeini, G. F. Cooper, and M. Hauskrecht, "Binary Classifier Calibration Using a Bayesian Non-Parametric Approach," in *Proc. SIAM Int. Conf. on Data Mining*, 2015, pp. 208–216.
- [36] F. Baumann *et al.*, "Sequential Boosting for Learning a Random Forest Classifier," in *IEEE Winter Conf. on Appl. of Comput. Vis.*, 2015, pp. 442–447.
- [37] B. Shahriari *et al.*, "Taking the Human Out of the Loop: A Review of Bayesian Optimization," *Proc. IEEE*, vol. 104, no. 1, pp. 148–175, 2015.
- [38] J. R. Villar *et al.*, "Generalized Models for the Classification of Abnormal Movements in Daily Life and its Applicability to Epilepsy Convulsion Recognition," *Int. J. of Neural Syst.*, vol. 26, no. 06, p. 1650037, 2016.
- [39] P. Schmidt *et al.*, "Introducing WESAD, a Multimodal Dataset for Wearable Stress and Affect Detection," in *Proc. Int. Conf. on Multimodal Interaction*, 2018, pp. 400–408.
- [40] N. Birbaumer *et al.*, "A Brain-Controlled Spelling Device for the Completely Paralyzed," *Nature*, pp. 297–298, 2001.
- [41] T. Theodoridis, "EMG Physical Action Data Set," UCI Machine Learning Repository, 2011, DOI: <https://doi.org/10.24432/C53W49>.
- [42] J. Friedman, T. Hastie, and R. Tibshirani, *The Elements of Statistical Learning*. Springer, 2001, vol. 1.
- [43] R. Tavenard *et al.*, "Tslern, A Machine Learning Toolkit for Time Series Data," *J. of Machine Learn. Res.*, vol. 21, no. 118, pp. 1–6, 2020. [Online]. Available: <http://jmlr.org/papers/v21/20-091.html>
- [44] A. Renault *et al.*, "Early Classification of Time Series: A Survey and Benchmark," *arXiv preprint arXiv:2406.18332*, 2024.
- [45] T. Instruments, "INA219 Zero-Drift, Bidirectional Current/Power Monitor with I²C Interface," 2015. [Online]. Available: <https://www.ti.com/product/INA219>
- [46] F. Pedregosa *et al.*, "Scikit-Learn: Machine Learning in Python," *J. of Machine Learning Res.*, vol. 12, pp. 2825–2830, 2011.



Sirine Belhaj received her B.S. degree in Multidisciplinary Engineering, with a specialization in Signals and Systems, from École Polytechnique de Tunisie, Tunis, Tunisia. She is currently a Computer Science PhD student at the School of Electrical Engineering and Computer Science at Washington State University, Pullman, WA, USA. Her research focuses on making machine learning systems more efficient for deployment on resource-constrained devices. She is a student member of IEEE.



Negar Haghpahanhi received her B.S. degree in Computer Science from Amirkabir University of Technology, Tehran, Iran. She is currently pursuing her Ph.D. degree in Computer Science at the School of Electrical Engineering and Computer Science, Washington State University, Pullman, WA, USA. Her research interests include machine learning, artificial intelligence, and efficient machine learning for resource-constrained computing systems. She is a student member of IEEE.



Lubah Nelson received the B.A. degree in Data Science from the University of California, Berkeley, Berkeley, CA, USA, and the M.S. degree in Computer Science from Washington State University, Pullman, WA, USA. She is an AI/ML Engineer and researcher with interests in AI, agentic systems, and resource-aware computing. Her current research focuses on efficient agentic orchestration and methods for reducing the computational and operational costs of large language models (LLMs).



Dina Hussein received her B.S. degree in Electrical Engineering from St. Cloud State University, Minnesota, USA, in 2020. She received her Ph.D. from the School of Electrical Engineering and Computer Science at Washington State University, Pullman, WA, USA in 2025. She is currently a Postdoctoral Researcher in the Department of Pediatrics at Wake Forest University School of Medicine.

Her research interests include applying machine learning to optimize the performance of wearable devices with a particular emphasis on health monitoring.



Ganapati Bhat is an Associate Professor in the School of Electrical Engineering and Computer Science at Washington State University. He received his B.Tech degree in Electronics and Communication from Indian Institute of Technology (ISM), Dhanbad, India in 2012 and the PhD degree in Computer Engineering from Arizona State University in 2020. His research interests include energy optimization in computing systems, dynamic thermal and power management, and mobile health systems.